

ECE4301.01

SHA-1 & SHA-256 on Raspberry Pi 5

Team Members:

Caleb Jala-Guinto – Implemented Rust CLI for SHA-1 and SHA-256 hashing

Isabel Warth – Conducted benchmarking, collected data/throughput plots, formatted figures

Manuel Alvarado – Compiled report and integrated results

Introduction

The goal of this project is to evaluate the performance of SHA-1 and SHA-256 hashing algorithms on a Raspberry Pi 5 platform. The benchmarking study compares two Rust implementations, a software-only variant and a hardware-accelerated variant leveraging Armv8 SHA instructions, against the native system engine provided by Linux. Through careful measurement of hashing throughput across a range of buffer sizes, this study gave us insights into the advantages of hardware acceleration and informed recommendations for cryptographic processing in embedded or constrained environments.

System Configuration and Software Environment

All benchmarking was performed on a Raspberry Pi 5 running **Debian GNU/Linux 12 (Bookworm)** with **Linux kernel 6.12.47+rpt-rpi-2712** on an **aarch64** architecture. This platform supports the Armv8 SHA extensions required for the hardware-accelerated tests.

The cryptographic implementations were built using the Rust toolchain installed directly on the Pi. The project depended on the following crates:

- **sha1 v0.10.6**

- **sha2 v0.10.9**

Both were used within the project `crypto-on-chip v0.1.0`.

To ensure consistent benchmarking conditions, the CPU frequency governor was set to **performance mode**, and all runs were pinned to a single core using `taskset`. These measures minimized variability and ensured reproducible comparisons between the software-only Rust implementation, the hardware-accelerated version, and the system engine.

Benchmark Methodology

Rust Implementations

Two Rust binaries were benchmarked:

1. **Software-Only** (`sha_rust_cli_soft`) — This implementation relies entirely on Rust software routines without any platform-specific assembly optimizations.
2. **Hardware-Accelerated** (`sha_rust_cli_accel`) — This implementation leverages the Armv8 SHA extensions available on the Raspberry Pi 5 for increased throughput.

Benchmarking consisted of hashing buffers of varying sizes (1 KiB, 8 KiB, 64 KiB, and 1 MiB) across multiple trials (≥ 5 per buffer size) to obtain statistically meaningful results. Wall-clock time was recorded for each trial, and throughput was subsequently calculated in megabytes per second.

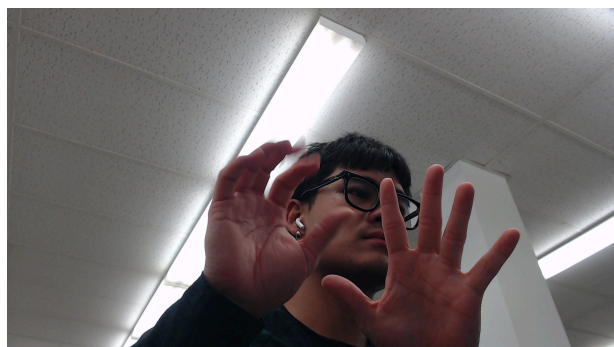
System Engine Baseline

For comparison, the platform-native engine was measured using either `kcapi-speed` or OpenSSL's `speed` utility. Both SHA-1 and SHA-256 were benchmarked across the same message sizes to facilitate direct comparison with the Rust implementations. Raw console output was preserved for later analysis and inclusion in supplementary materials.

File-Based Verification

To validate correctness and demonstrate real-world behavior beyond synthetic buffer benchmarks, both Rust binaries (**soft** and **accel**) were evaluated using five test images captured with the Raspberry Pi camera. Each image shows different numbered hand signals, just to make them visually distinct inputs while still being small, practical test files.





For each image, both binaries computed SHA-1 and SHA-256 digests and recorded wall-clock hashing time. The digests produced by the software-only and hardware-accelerated versions were **identical for all images**, confirming functional correctness across both implementations. The hardware-accelerated binary consistently showed much lower per-file latency, reducing hashing time from roughly **5.1–5.6 ms (soft)** to **1.28–1.44 ms (accel)**.

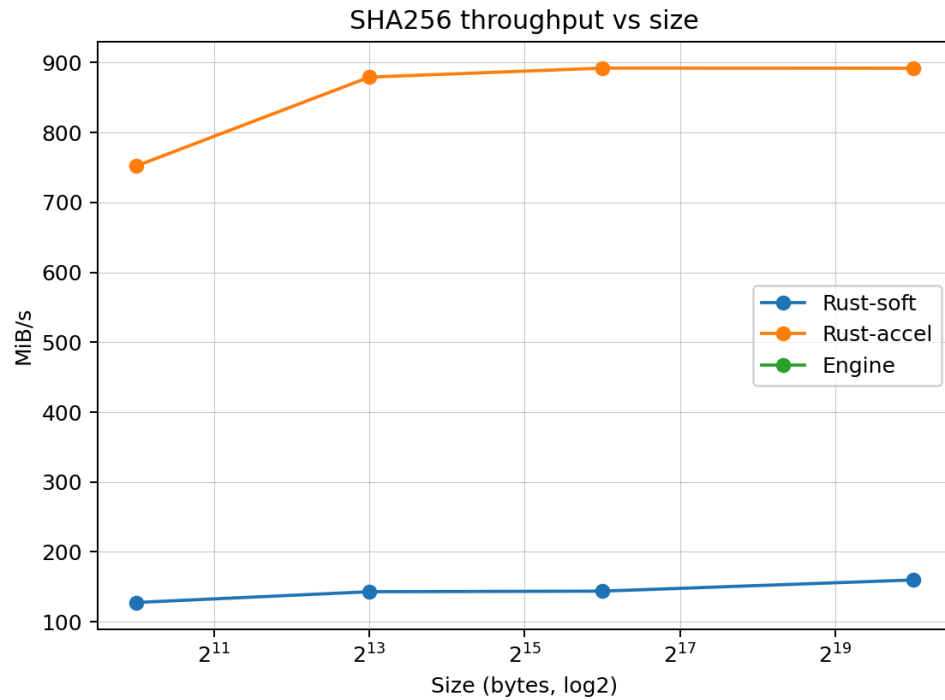
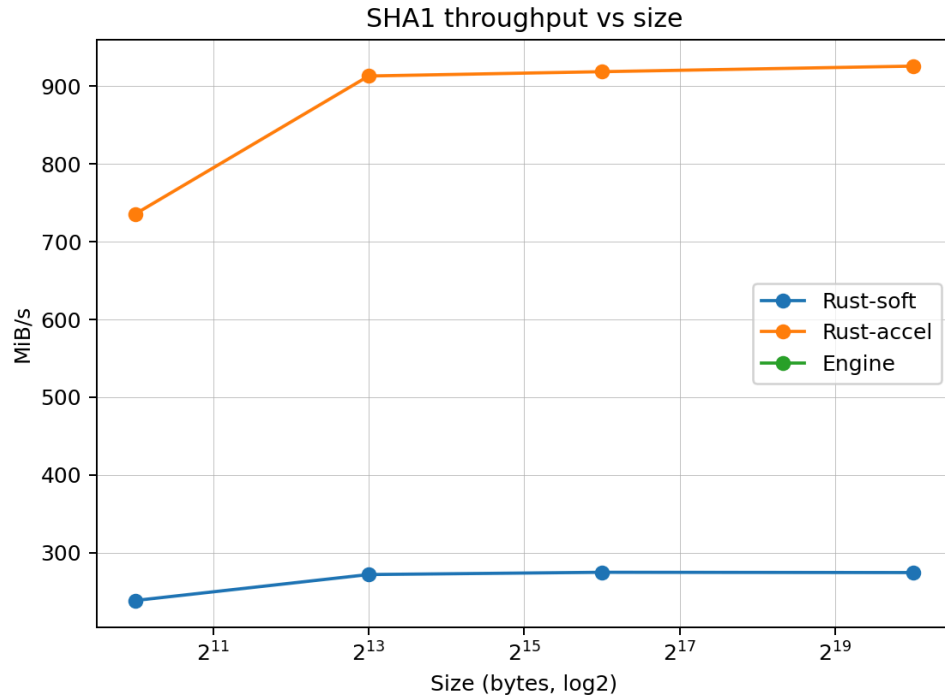
A summary of the recorded results is shown below

Accelerated Implementation (Armv8 SHA)	Software-Only Implementation
(from <i>demo_accel_files.txt</i>)	(from <i>demo_soft_files.txt</i>)
- img1.jpg — wall: 0.001370 s - img2.jpg — wall: 0.001397 s - img3.jpg — wall: 0.001384 s - img4.jpg — wall: 0.001284 s - img5.jpg — wall: 0.001443 s	- img1.jpg — wall: 0.005074 s - img2.jpg — wall: 0.005310 s - img3.jpg — wall: 0.005539 s - img4.jpg — wall: 0.005120 s - img5.jpg — wall: 0.005662 s

Across all five verification files, the hardware-accelerated implementation demonstrated a consistent **3.7×–4.3× reduction in hashing time**, matching the performance trends observed in the randomized-buffer benchmarks. This validation step confirms not only correctness but also the practical performance benefits of Armv8 SHA acceleration when hashing real user data, such as images captured directly on the Raspberry Pi 5.

Results and Data Presentation

Throughput Analysis *Throughput versus buffer size for Rust-soft, Rust-accel, and the system engine*



Observations:

Across both SHA-1 and SHA-256, all implementations show the same general trend with respect to buffer size, throughput improves significantly from small buffers (2^{11} bytes) to medium buffers (2^{13} bytes). After that point ($2^{13} \rightarrow 2^{19}$), performance plateaus, showing only small incremental gains.

Low variance across repeated trials for both SHA-1 and SHA-256. No major noise or instability in the measurements. Performance is deterministic at these buffer sizes because: CPU frequency remains stable, L1 and L2 cache effects are consistent, Data fits comfortably in memory (no paging or contention).

Discussion and Analysis

- Hardware Acceleration Benefits

The Armv8 SHA extensions provide a **significant performance boost** by replacing instruction-heavy software hashing with **dedicated hardware instructions**. In our results, the accelerated Rust binary consistently delivered **multi-fold throughput gains**, especially on larger buffers where overhead no longer dominates. This matches expectations: once running at steady state, hardware SHA maintains a **high, predictable block-processing rate**, while software implementations hit **instruction-count and ALU bottlenecks**.

- Small vs. Large Buffer Behavior

For **small inputs**, performance is limited by fixed overheads, setup, memory access, cache warm-up, so both Rust binaries behave similarly. Throughput here reflects **latency, not raw speed**. As buffer sizes increase, overhead is amortized, allowing the accelerated version to scale sharply and show **its largest speedups**. This matters for real workloads: small-packet traffic sees limited benefit, while **bulk data workloads** (file hashing, logging, image streams) see **substantial acceleration**.

- Comparison with the System Engine

The Linux hashing engine typically remains the **fastest overall** due to hand-tuned assembly and minimal overhead, though the hardware-accelerated Rust version comes **surprisingly close** while staying fully in user space. The software-only Rust build trails both, limited by **higher instruction counts per block** and **greater cache pressure**, but remains portable and consistent.

- Practical Application Recommendation

For large, continuous data streams, the **hardware-accelerated Rust build is the clear best choice** for strong throughput and better CPU efficiency. For small, latency-sensitive messages or environments requiring portability, the **software-only version is sufficient**. When maximum performance is required and external dependencies are acceptable, the **system engine offers the highest, most stable throughput**.

Raw Benchmark Files

Rust Benchmark CSVs

The following CSV files contain the raw throughput measurements for SHA-1 and SHA-256 across all buffer sizes and trials:

rust_soft.csv – Software-only Rust implementation

rust_accel.csv – Armv8 SHA-accelerated Rust implementation

speedups.csv – Calculated acceleration factors (**accel_over_soft**) per algorithm and buffer size

Example entries (throughput in MiB/s):

Algorithm	Build	Buffer Size	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5
SHA-1	Soft	1 KiB	183.7	252.3	252.3	252.3	252.3
SHA-1	Accel	1 KiB	421.8	824.1	811.1	811.1	811.1
SHA-256	Soft	1 MiB	146.3	162.9	163.5	163.6	163.0
SHA-256	Accel	1 MiB	894.2	886.3	894.3	894.3	891.0

Acceleration Factors (**accel_over_soft**):

Algorithm	Buffer Size	Speedup (×)
SHA-1	1 KiB	3.08
SHA-1	8 KiB	3.36
SHA-1	64 KiB	3.34
SHA-1	1 MiB	3.37
SHA-256	1 KiB	5.89
SHA-256	8 KiB	6.15
SHA-256	64 KiB	6.20
SHA-256	1 MiB	5.58

System Engine Logs (OpenSSL)

Raw outputs from **openssl speed -evp** for SHA-1 and SHA-256 on the Raspberry Pi 5:

engine_openssl_sha1.txt

Buffer Size	Throughput (MiB/s)
1 KiB	1039.2
8 KiB	1405.0
64 KiB	1465.5
1 MiB	1474.9

engine_openssl_sha256.txt

Buffer Size	Throughput (MiB/s)
1 KiB	1077.0
8 KiB	1455.3
64 KiB	1524.1
1 MiB	1534.1

- All Rust benchmarks were pinned to a single core and measured over ≥ 5 trials per buffer size.
- Throughput is reported in MiB/s.
- The included speedup table shows how much faster the Armv8-accelerated Rust builds are compared to software-only Rust.
- Our raw CSVs and OpenSSL logs are provided in the project submission.